



Universite Sultan Moulay Slimane

ENSA Beni Mellal ★ Departement Informatique

Rapport de TP N°2

Architecture Distribuee

Node.js

Docker

MongoDB

Redis

RabbitMQ



Module : Architecture des Systemes Distribues



Realisee par : Rayhana Laznasni



Encadre par : Pr. BE ELBAGHAZAOUI



Annee : 2025 – 2026



Systemes Distribues ★



Docker ★



Microservices

Table des matières

1	Introduction	2
2	Architecture cible	2
2.1	Composants	2
2.2	Flux de données	2
3	Tests avec Postman	3
3.1	Test 1 – Créer une commande (POST)	3
3.2	Test 2 – Observer le cache Redis (GET)	5
3.3	Test 3 – Observer le load balancing	6
4	Reponses aux questions	8
5	Conclusion	9
6	References	9

1 Introduction

Dans le cadre du module **Architecture des Systemes Distribues**, ce TP2 deploie une architecture distribuee complete simulant un systeme e-commerce reel avec **6 composants** interconnectes.

Flux complet implemente :



L'architecture comprend :

- Un **load balancer** Nginx
- Deux instances Node.js
- Une base MongoDB
- Un cache Redis
- Un bus de messages RabbitMQ

2 Architecture cible

2.1 Composants

Composant	Technologie	Role
Client	Postman	Envoie les requetes HTTP POST/GET
Load Balancer	Nginx	Repartit le trafic (Round-robin)
Serveur applicatif	Node.js	Implemente la logique metier
Base de donnees	MongoDB	Stocke les commandes
Cache	Redis	Memorise les GET (TTL 30s)
Bus de messages	RabbitMQ	Notifications asynchrones

TABLE 1 – Composants de l'architecture distribuee

2.2 Flux de donnees

Listing 1 – Flux de l'architecture

```

1 Postman --> Nginx :80 --> Node.js :3001 / :3002
2 Node.js --> MongoDB (stockage persistant)
3 Node.js --> Redis (cache GET /commandes)
4 Node.js --> RabbitMQ --> Consommateur --> Notification
  
```

3 Tests avec Postman

3.1 Test 1 – Créer une commande (POST)

Parametre	Valeur
Methode	POST
URL	http://localhost/commandes
Headers	Content-Type: application/json
Body	{"produit": "Laptop Dell XPS", "quantite": 2, "client": "Ahmed Benali"}
Reponse	201 Created + champ traitee_par

TABLE 2 – Test POST /commandes

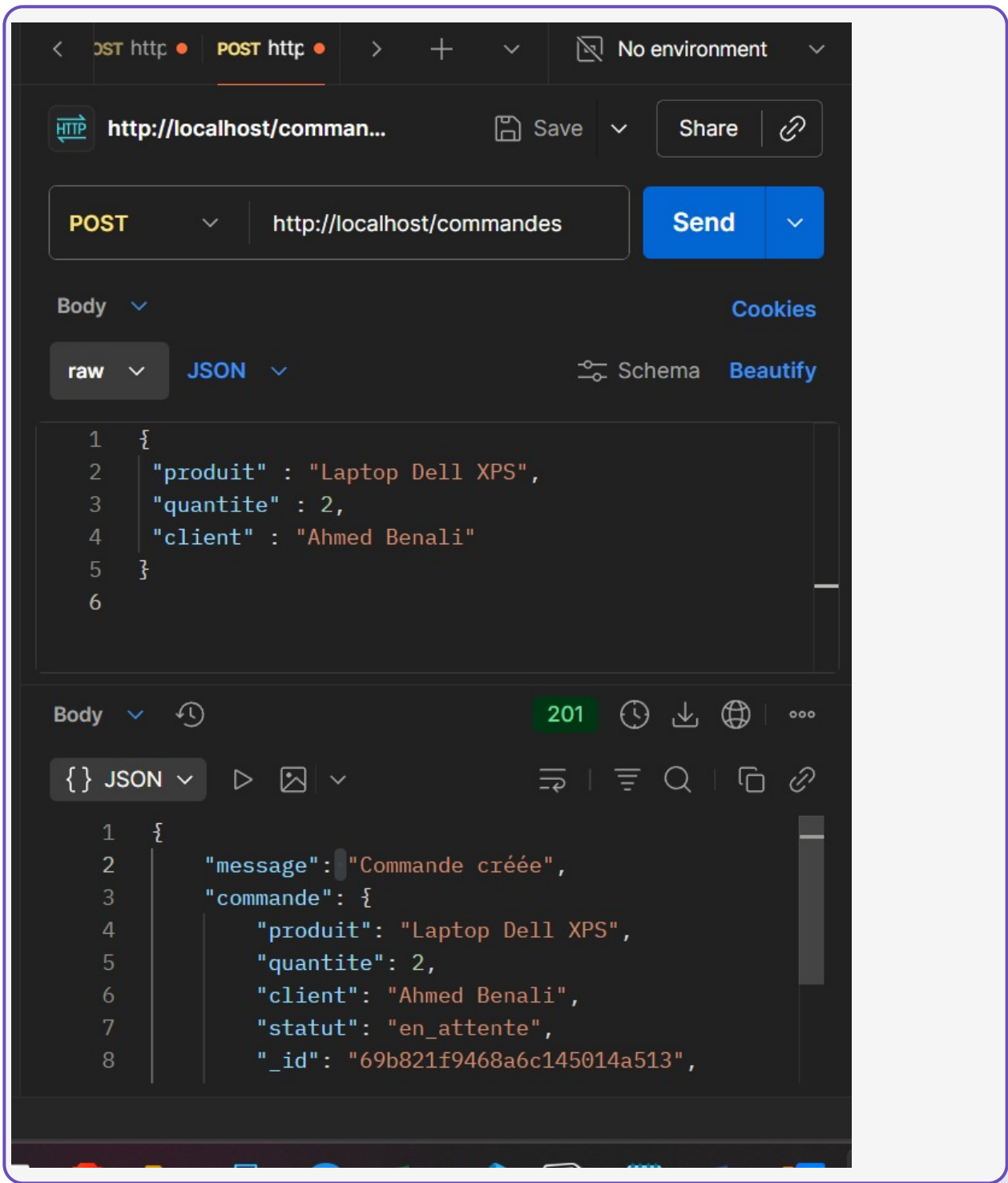


FIGURE 1 – 📷 Postman – POST /commandes – reponse 201

The terminal window displays MongoDB logs for a checkpoint saving operation. The logs show a 'WT_SESSI ON.checkpoint' message with details like 'category_id:6', 'verbose_level:DEBUG', and 'saving checkpoint snapshot min: 374, snapshot max: 374 snapshot count: 0, oldest timestamp: (0, 0)'. The Docker Desktop interface on the right shows the 'app2' instance running on port 3002, with options to 'View in Docker Desktop', 'View Config', and 'Enable'.

FIGURE 2 – ➤ Terminal – MongoDB sauvegarde + message publie dans RabbitMQ

3.2 Test 2 – Observer le cache Redis (GET)

- 1er appel GET : source = MongoDB
- 2eme appel GET (dans les 30s) : source = CACHE Redis

💡 **Observation :** La 2eme reponse est plus rapide car elle provient du cache Redis sans interroger MongoDB.

The screenshot shows a Postman GET request to 'http://localhost/commandes'. The response is a 200 OK status with a response time of 313 ms and a body size of 953 B. The response body is in JSON format and contains an array of command objects. The first object in the array has the following fields: 'source' (MongoDB), 'traitee_par' (Instance-1), 'commandes' (an array of objects), 'id' (69b821f9468a6c145014a513), 'produit' (Laptop Dell XPS), 'quantite' (2), 'client' (Ahmed Benali), 'statut' (en_attente), 'date' (2026-03-16T15:30:01.931Z), and 'v' (0).

FIGURE 3 – 📷 Postman – GET /commandes – source MongoDB puis source CACHE Redis

3.3 Test 3 – Observer le load balancing

En envoyant plusieurs requetes POST successives, on observe l'alternance **Round-robin** entre Instance-1 et Instance-2.

Requete	traitee_par
Requete 1	Instance-2
Requete 2	Instance-1
Requete 3	Instance-2
Requete 4	Instance-1

TABLE 3 – Observation du Round-robin Nginx

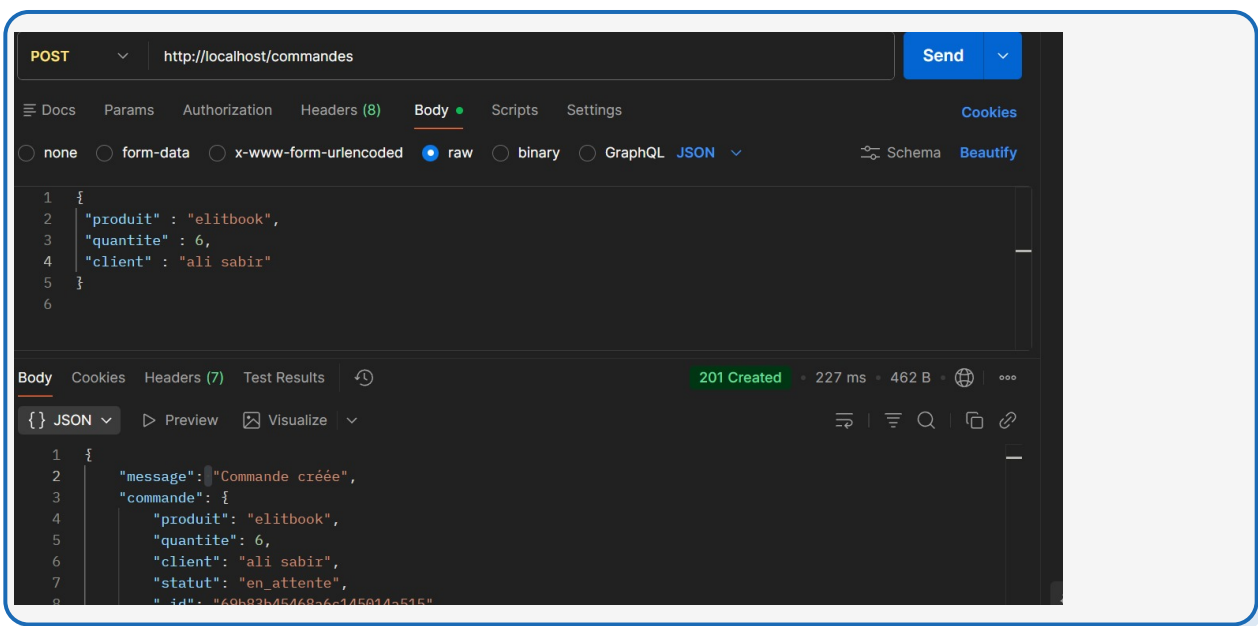


FIGURE 4 – 📷 Postman – Requete 1 traitee par Instance-1

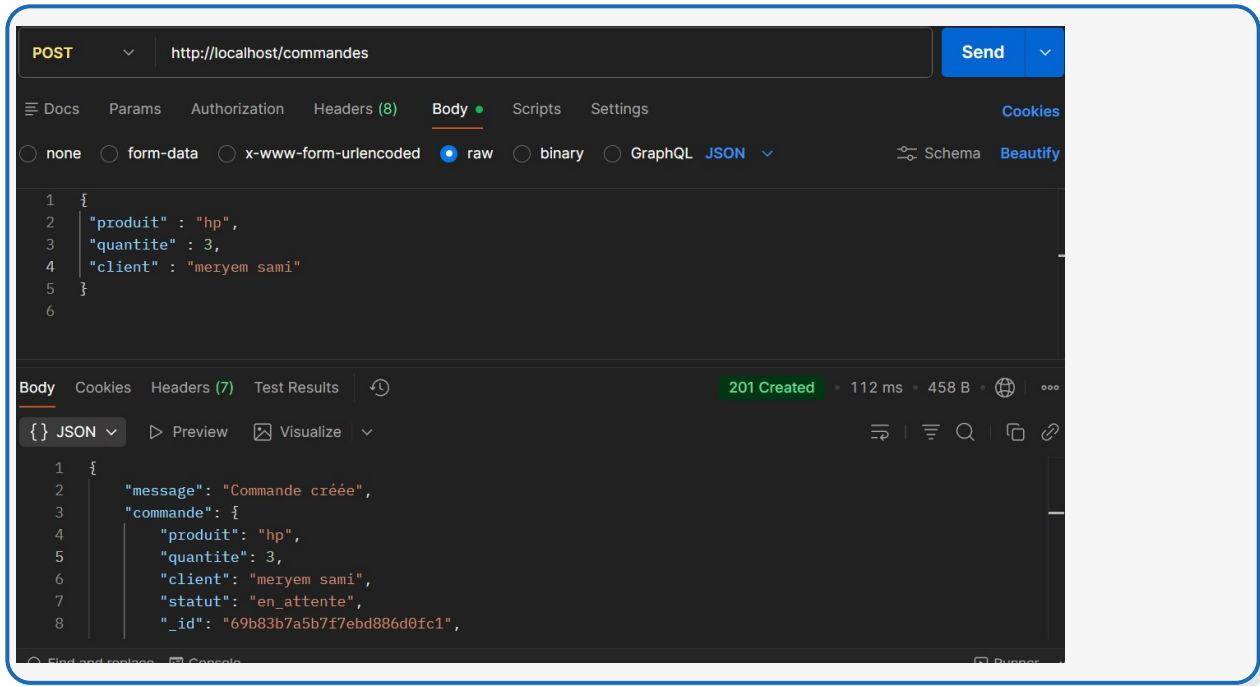


FIGURE 5 – 📷 Postman – Requete 2 traitee par Instance-2

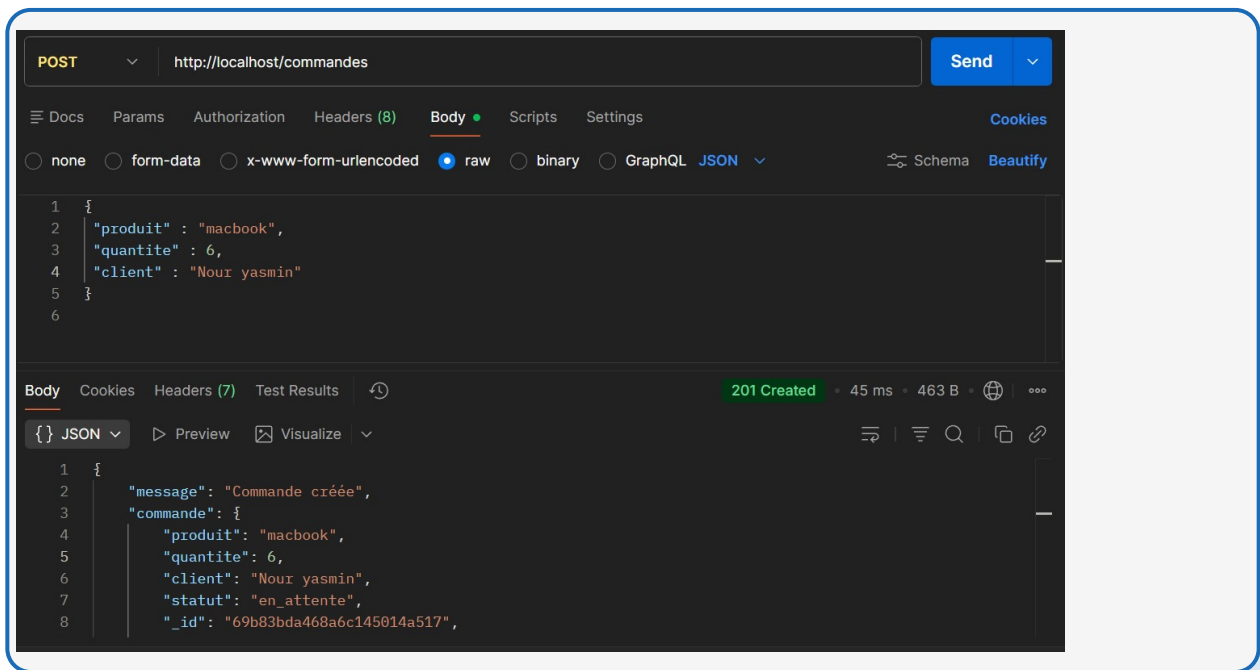


FIGURE 6 – 📷 Postman – Requete 3 traitee par Instance-1 – Round-robin confirme


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE
"POST /commandes HTTP/1.1" 201 223 "-" "PostmanRuntime/7
.51.1"
mongodb | {"t":{"$date":"2026-03-16T17:20:55.961+00:00
"}, "s":"I", "c":"WTCHKPT", "id":22430, "ctx":"Checkp
ointer", "msg":"WiredTiger message", "attr":{"message":{"t
s_sec":1773681655, "ts_usec":961419, "thread":"1:0x74b130d
47640", "session_name":"WT_SESSION.checkpoint", "category"
:"WT_VERB_CHECKPOINT_PROGRESS", "category_id":6, "verbose_
level":"DEBUG", "verbose_level_id":1, "msg":"saving checkp
oint snapshot min: 576, snapshot max: 576 snapshot count
: 0, oldest timestamp: (0, 0), meta checkpoint timestam
p: (0, 0) base write gen: 54"}}}
mongodb | {"t":{"$date":"2026-03-16T17:21:56.007+00:00
"}, "s":"I", "c":"WTCHKPT", "id":22430, "ctx":"Checkp
ointer", "msg":"WiredTiger message", "attr":{"message":{"t
s_sec":1773681716, "ts_usec":7177, "thread":"1:0x74b130d47
640", "session_name":"WT_SESSION.checkpoint", "category":"
WT_VERB_CHECKPOINT_PROGRESS", "category_id":6, "verbose_le
vel":"DEBUG", "verbose_level_id":1, "msg":"saving checkpoi
nt snapshot min: 581, snapshot max: 581 snapshot count:
0, oldest timestamp: (0, 0), meta checkpoint timestamp:
(0, 0) base write gen: 54"}}}
app2 | at done (/app/node_modules/raw-body/index.js
:227:7)
app2 | at IncomingMessage.onEnd (/app/node_modules/
raw-body/index.js:287:7)
app2 | at IncomingMessage.emit (node:events:517:28)
app2 | at endReadableNT (node:internal/streams/read
app2 | at IncomingMessage.onEnd (/app/node_modules/
raw-body/index.js:287:7)
app2 | at IncomingMessage.emit (node:events:517:28)
app2 | at endReadableNT (node:internal/streams/read
app2 | at IncomingMessage.emit (node:events:517:28)
app2 | at endReadableNT (node:internal/streams/read
app2 | at process.processTicksAndRejections (node:i
nternal/process/task_queues:82:21)
app2 | Message publié dans RabbitMQ
app1 | Message publié dans RabbitMQ
app2 | Message publié dans RabbitMQ

```

FIGURE 7 – Terminal – alternance Instance-1 / Instance-2 dans les logs

4 Reponses aux questions

Q1 – Quel composant recoit en premier les requetes ?

Nginx recoit toutes les requetes sur le port 80 et utilise l'algorithme **Round-robin** : req1 → app1, req2 → app2, req3 → app1, etc.

Q2 – Que se passe-t-il si app1 tombe ?

Nginx **continue de fonctionner** et redirige tout le trafic vers **app2**. C'est le principe de **haute disponibilité**.

Q3 – Difference 1er GET vs 2eme GET ?

- **1er GET** : cache vide → MongoDB → stocke dans Redis (TTL 30s) → source: MongoDB
- **2eme GET** : donnees en cache → reponse Redis instantanee → source: CACHE Redis

Q4 – Pourquoi invalider le cache apres un POST ?

Sans invalidation, les GET retourneraient les **anciennes donnees**. `redis.del('toutes_commandes')` force MongoDB a recreer un cache a jour, garantissant la **coherence des donnees**.

Q5 – RabbitMQ et la resilience ?

- **Decouplage** : traitement **asynchrone**
- **Persistence** : messages conserves en queue
- **Scalabilite** : plusieurs consommateurs en parallele

+ Q6 – Ajouter une 3eme instance ?

Modifier **deux fichiers** :

1. `docker-compose.yml` : ajouter service `app3` avec `PORT=3003`
2. `nginx.conf` : ajouter dans le bloc `upstream` :

```
upstream backend {
    server app1:3001;
    server app2:3002;
    server app3:3003;
}
```

5 Conclusion

Ce TP nous a permis de deployer une **architecture distribuee complete** avec 6 composants. Principaux acquis :

- Role du **Load Balancer** Nginx et Round-robin
- Orchestration avec **Docker Compose**
- Optimisation via **cache Redis**
- Traitement asynchrone avec **RabbitMQ**
- Observation de la **repartition de charge**

6 References

- Documentation Node.js
- Docker Compose
- Nginx
- MongoDB
- Redis
- RabbitMQ